Skyline

Frontend & Integration Guide | Beginner-friendly project map

1. Frontend architecture

What is actually in this repository? The UI is React 18 built with Vite 5. It uses React Router for URLs, Lucide React icons and the qrcode package for QR rendering. The API is an Express 5 + TypeScript Node server. Its active business data store is a JSON file, not the configured Prisma/PostgreSQL connection.

How the app starts

```
frontend/src/main.jsx -> BrowserRouter + AuthProvider -> App.jsx
backend/src/index.ts -> Express app from src/app.ts -> listen on configured port (default 5000)
```

Routes and role-based dashboards

URL	Page / access
/ /events /merchandise /signin	Public dashboard variants; event and product detail pages use route parameters.
/admin	AdminDashboard, wrapped in ProtectedDashboard for role admin.
/member	MemberDashboard, role member.
/core-member	CoreMemberDashboard, role core_member.

React terms in this project

- **Component:** reusable UI function, such as `AdminEvents` or `MyTickets`. Dashboards compose components into sections.
- **Props:** values passed by a parent; for example `MyTickets` receives its ticket list as `tickets`.
- **State:** component-owned changing data (form fields, current tab, open modal). Hooks such as `useState` and `useEffect` implement it.
- **Role check:** `ProtectedDashboard` redirects users without the required UI role. This is a frontend gate, not backend API security.

Folders worth knowing

`frontend/src/dashboards/` contains public, admin, member and core-member pages; `components/` contains shared UI/payment components; `services/api/skylineApi.js` is the shared API client; `services/auth/` handles session UI state. Backend routes live in `backend/src/routes/`; data types and persistence live in `backend/src/services/store.ts`.

2. How frontend integration works

A request crosses the browser/server boundary using HTTP, the standard request/response protocol between frontend and backend.

```
React component -> skylineApi method -> fetch(HTTP) -> Express route
-> store/service logic -> backend/data/db.json -> JSON response
-> component updates state -> React redraws the screen
```

The shared API client

`frontend/src/services/api/skylineApi.js` wraps browser `fetch`. It builds the API base URL from `VITE_API_BASE_URL` (or the code default), adds JSON headers, parses JSON responses, and throws an error for non-2xx responses. It also exposes `notifyDataRefresh` / `subscribeToDataRefresh`, a browser event used by dashboards to reload after changes.

Real example: creating an event

- **1. Form:** an admin enters details in `AdminEvents.jsx`.

- **2. API client:** it calls `skylineApi.createEvent(form)`, which sends `POST /events`. The API base prefix makes this `POST /api/events`.

- **3. Route:** Express mounts `eventsRouter` at `/api/events`. Its handler checks title/date and creates an event object.

- **4. Persistence:** the handler inserts into `store.db.events` and calls `store.commit()`, which writes the JSON store file.

- **5. UI refresh:** the route returns JSON with the new event; the component reloads data and emits a refresh event so dashboard sections can re-fetch.

Authentication: what the code does (and does not do)

`AuthContext` stores the current user in React. `authService.js` calls `POST /api/auth/login`, then stores the username in sessionStorage under `skyline-demo-session`. The API client sends it as an Authorization Bearer value and `X-User-Id`. In the backend, `GET /api/auth/session` interprets that value as a username. This is a demo session marker, not a signed token. The dashboard role guard is in the browser; route handlers do not use shared authentication middleware to authorize every request.

Not production-grade authorization: the code does not establish a tamper-resistant token or centrally enforce admin permissions on every API route.

Which database is really used?

For routes inspected, `store.ts` synchronously reads/writes `backend/data/db.json`. Handlers mutate the in-memory object then call `commit()`. Prisma/PostgreSQL configuration and a Prisma client exist, but business routes import the JSON store instead. This finding describes source wiring, not the data/configuration of a deployed environment.

3. Features and payment flow

This table maps the UI to the actual route and persistence mechanism inspected in this repository.

Feature	Frontend	Backend endpoint(s)	Persistence
Events / counts	admin/components/AdminEvents.jsx AdminDashboard.jsx	GET, POST /api/events PUT, DELETE /api/events/:id	store.db.events in db.json
Member directory	admin/components/AdminMembers.jsx	GET /api/admin/members PUT /:id/status, /:id/role DELETE /:id	store.db.users in db.json
Reimbursements	member/components/MyReimbursements.jsx admin/components/AdminReimbursements.jsx	GET, POST /api/finance/reimbursements PUT /api/finance/reimbursements/:id/status	store.db.reimbursements in db.json
Announcements	admin/components/AdminAnnouncements.jsx	GET, POST /api/announcements	store.db.announcements in db.json
Event passes / QR	member/components/MyTickets.jsx components/PaymentCheckoutModal.jsx	GET /api/events/tickets/my-tickets POST /api/events/tickets/verify-qr	store.db.tickets + event metadata
Checkout	components/PaymentCheckoutModal.jsx	POST /api/payments/checkout-session POST /api/payments/verify-payment	Orders/tickets, stock, transactions in JSON store

Payment sequence in this build

- `PaymentCheckoutModal` asks `POST /api/payments/checkout-session` for a server-calculated amount and sandbox session.
- On Pay, the modal calls `POST /api/payments/verify-payment`. The route creates the order or ticket, changes stock/event registration, records a transaction where applicable, commits the JSON store and returns success.
- Only a successful response is treated as confirmed. The modal then triggers a UI refresh and displays the returned order/ticket.

Important: this is not an integrated Razorpay checkout.

The UI labels the provider "Skyline Sandbox Gateway"; the backend generates IDs beginning `CS-SANDBOX-` and a sandbox client secret. No Razorpay SDK flow or Razorpay signature verification is present in the inspected checkout path. The verify endpoint accepts a session ID and fulfills the purchase; it does not verify a provider-signed payment callback. This is a simulated flow, not proof of a real captured payment.

Mock data and incomplete integration

The admin and member dashboards import initial datasets from `adminMockData.js` and `memberMockData.js`. They try live API requests, but some loaders retain prior state when a request fails; `MyReimbursements` can also fall back to initial claims. A displayed value may therefore be stale/mock when loading fails. The data files contain illustrative records, including attendee data; the inspected event routes do not provide a matching attendees endpoint. The JSON store is local-file persistence, not a shared relational database.

4. Findings, run commands, and explanation

Use these notes to distinguish working code paths from production assumptions.

Key integration findings

- **Data source:** runtime business routes use `backend/data/db.json` via `store.ts`. Prisma/Supabase config is present but is not wired into the inspected route handlers.

- **Security gap:** the bearer value is a username, not a signed token; UI role checks do not replace backend authorization. Inspected routes are not protected by shared auth middleware.

- **Payment gap:** checkout is explicitly a sandbox simulation; no Razorpay client/callback/signature verification was found in this path.

- **Mock visibility:** dashboards initialize from mock datasets and may keep old data after API failures. Verify the network response before treating a number as live.

- **Shared data limits:** concurrent writes, multiple server instances and durable production operation are not addressed by a database transaction layer.

Run locally

Open two terminals from the repository workspace. Install dependencies if they are not already present.

```
Terminal A - backend
cd backend
npm install
npm run dev

Terminal B - frontend
cd frontend
npm install
npm run dev
```

The backend script uses `tsx watch` and defaults to port 5000 in `backend/src/config/env.ts`. Vite serves the frontend using its configured/default development port. `skylineApi` defaults to `http://localhost:5000/api`; configure the API base if the backend is elsewhere. Available production checks are `npm run build` and, in backend, `npm test`.

Architecture at a glance

React UI components + dashboards	**HTTP client** skylineApi / fetch	**Express API** routes + handlers	**JSON store** store.commit()

UI action -> API request -> route logic -> JSON file -> JSON response -> state refresh

Takeaways for a demo or viva

- React dashboards are role-specific views assembled from reusable components.

- `skylineApi` centralizes browser-to-Express calls; routes process requests and return JSON.

- In this repository, the working persistence path is the JSON file, not Prisma/PostgreSQL.

- The payment experience is sandbox-only; call it simulated checkout, not a Razorpay integration.

- Mock fallback data, frontend-only role checks and local-file storage are limitations, not production guarantees.

Verified from source files in this workspace. Endpoints and implementation statements refer to checked code; deployed settings and live database behavior were not independently verified.